MCE5919: Gaussian Process for Machine Learning and Signal Processing

Lecture 6

Wenqiang PU

Shenzhen Research Institute of Big Data & School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

March 18, 23, 2026

Notation

- Training data: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, inputs $\mathbf{x}_i \in \mathbb{R}^D$, outputs $y_i \in \mathbb{R}$.
- Kernel (covariance function): $k(\mathbf{x}_p, \mathbf{x}_q; \boldsymbol{\theta})$ with hyperparameter vector $\boldsymbol{\theta}$.
- Kernel (Gram) matrix: $K_f = k(X, X; \boldsymbol{\theta}) \in \mathbb{R}^{n \times n}$, $[K_f]_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.
- Noisy kernel matrix: $K_y = K_f + \sigma_n^2 I$.
- Log marginal likelihood: $\mathcal{L}(\boldsymbol{\theta}) = \log p(\mathbf{y} \mid X, \boldsymbol{\theta})$.
- $\boldsymbol{\alpha} = K_y^{-1}\mathbf{y}$: "dual coefficients" used throughout.
- Model structure: $\mathcal{H}_i$ (e.g., SE kernel, Matérn kernel, periodic kernel).

Overview

1. Model Selection Problems

2. Bayesian Model Selection

3. Cross-Validation for Model Selection

4. Automatic Relevance Determination

5. Model Training for GP Regression/Classification
   5.1 GP Regression Training
   5.2 GP Classification Training

What is Model Selection?

We use the term model selection to cover both discrete choices (e.g., which kernel to use) and the setting of continuous-valued hyper-parameters

In particular, model selection serves two purposes:

- Refining the predictions of the model (better generalisation).
- Extracting interpretation about the properties of the data.

Remark

We will refer to the selection of a kernel function and its hyper-parameters as the training of a Gaussian process.

Underfitting and Overfitting

Underfitting (high bias)

Model is too simple to capture the underlying structure in the data.

- Both training error and test error are high.
- Example for GP: using a very long length-scale $\ell \to \infty$ (near-constant function).

Overfitting (high variance)

Model is too complex and memorizes noise in the training data.

- Training error is very low; test error is high.
- Example for GP: using an extremely short length-scale $\ell \to 0$ (interpolates every noisy observation).

Goal of model selection

Find the optimal hyperparameters that balance model fit and model complexity, achieving the best generalisation performance on unseen data.

General Approaches to Model Selection

Frequentist / Resampling

- Hold-out validation
- K-fold cross-validation (CV)
- Leave-one-out CV (LOO-CV)
- AIC / BIC

Bayesian

- Marginal likelihood (evidence)
- Bayesian model comparison
- Type-II maximum likelihood

Three general principles

1. Computing the probability of the model given the data (Bayesian approach).
2. Estimating the generalisation error (cross-validation).
3. Bounding the generalisation error (PAC-Bayes, VC theory).

Summary of Model Selection Problems

- Model selection is essentially open ended.
- There are endless variations in model selection.
- The following general principles hold mostly:
  1. Computing the probability of the model given the data;
  2. Estimating the generalization error;
  3. Bounding the generalization error.
  4. Remark: the generalization error means the average error on unseen test examples.

Overview

1. Model Selection Problems

2. Bayesian Model Selection

3. Cross-Validation for Model Selection

4. Automatic Relevance Determination

5. Model Training for GP Regression/Classification
   5.1 GP Regression Training
   5.2 GP Classification Training

Bayesian Model Selection

It is common to use a hierarchical specification of models.

- At the low level are the **hyper-parameters** $\boldsymbol{\theta}$ that control the covariance structure.
- At the top level we may have a (discrete) set of possible model structures, $\mathcal{H}_i$, under consideration.

Inference takes place one level at a time, by applying the rules of probability theory.

GP Regression

- $\boldsymbol{\theta} = \{\ell, \sigma_f^2, \sigma_n^2\}$ (SE kernel)
- $\mathcal{H}_i$: SE vs. Matérn vs. Periodic kernel

GP Classification

- Same $\boldsymbol{\theta}$ structure; non-Gaussian likelihood
- $\mathcal{H}_i$:SE vs. Matérn vs. Periodic kernel

Level 1: Posterior over Hyper-parameters

- The marginal likelihood from Level 1 plays the role of the likelihood at this level:

$$p(\boldsymbol{\theta}|\mathbf{y}, X, \mathcal{H}_i) = \frac{p(\mathbf{y}|X, \boldsymbol{\theta}, \mathcal{H}_i)\, p(\boldsymbol{\theta}|\mathcal{H}_i)}{p(\mathbf{y}|X, \mathcal{H}_i)} \tag{1}$$

  where $p(\boldsymbol{\theta}|\mathcal{H}_i)$ is the hyper-prior.

- The normalizing constant is given by

$$p(\mathbf{y}|X, \mathcal{H}_i) = \int p(\mathbf{y}|X, \boldsymbol{\theta}, \mathcal{H}_i)\, p(\boldsymbol{\theta}|\mathcal{H}_i)\, d\boldsymbol{\theta}. \tag{2}$$

- Empirical Bayes: Instead of computing the integral in eq. (2) (often intractable), we maximize the marginal likelihood

Level 2: Posterior over Model Structures

- At the top level, we compute the posterior for each model structure:

$$p(\mathcal{H}_i|\mathbf{y}, X) = \frac{p(\mathbf{y}|X, \mathcal{H}_i)\, p(\mathcal{H}_i)}{p(\mathbf{y}|X)}, \tag{3}$$

  where $p(\mathbf{y}|X) = \sum_i p(\mathbf{y}|X, \mathcal{H}_i)\, p(\mathcal{H}_i)$.

Bayes factor

Two models with equal prior probability, the posterior odds equals the Bayes factor:

$$\frac{p(\mathcal{H}_1|\mathbf{y}, X)}{p(\mathcal{H}_2|\mathbf{y}, X)} = \frac{p(\mathbf{y}|X, \mathcal{H}_1)}{p(\mathbf{y}|X, \mathcal{H}_2)}.$$

Note

- Evaluating the integral can be very cumbersome in practice.
- As an approximation, maximize the marginal likelihood w.r.t. $\boldsymbol{\theta}$.

Occam's Razor and the Marginal Likelihood

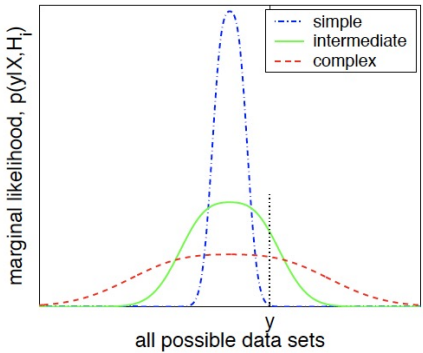


Figure: The marginal likelihood $p(\mathbf{y}|X, \mathcal{H}_i)$ for three models of different complexity. The intermediate-complexity model achieves the highest value at the observed $\mathbf{y}$ (dashed line).

Occam's Razor

Occam's razor

This effect is called Occam's razor after William of Occam 1285-1349, whose principle: "plurality should not be assumed without necessity" he used to encourage simplicity in explanations.

Why does the marginal likelihood prefer simpler models?

Since the marginal likelihood must normalise to unity, a complex model spreading probability over a wider range of datasets will have a lower peak value for any specific $\mathbf{y}$.

- A simple model: only account for a limited range of possible sets of target values, the model have a large value of the marginal likelihood.
- A complex model: it is capable of accounting for a wider range of data sets, and consequently the marginal likelihood does not attain such large values as for the simple model.

Overview

1. Model Selection Problems

2. Bayesian Model Selection

3. Cross-Validation for Model Selection

4. Automatic Relevance Determination

5. Model Training for GP Regression/Classification
   5.1 GP Regression Training
   5.2 GP Classification Training

Cross-Validation

In this section we introduce cross-validation (CV) for model selection.

Hold-out CV

The basic idea is to split the training set into two disjoint sets:

- Training set: used for model training;
- Validation set: used to monitor prediction performance.

The performance on the validation set is used as a proxy for the generalisation error and model selection is carried out based on this measure.

K-Fold Cross-Validation

Drawback of hold-out CV

Only a fraction of the full dataset can be used for training. If the validation set is small, the performance estimate may have large variance.

K-fold CV

To remedy the above problem:

- Split the data into k disjoint, equally-sized subsets: $\{X_1, X_2, \ldots, X_k\}$.
- For $i = 1, \ldots, k$: leave out X_i for validation; train on the remaining $k - 1$ subsets.
- Average the validation performance over the k runs.

A typical value is $k \in [5, 10]$.

Note

Computational cost: k full model training runs.

Leave-one-out cross-validation (LOO-CV)

- An extreme case of K-fold CV:
  For $k = n$, the number of training cases, also known as leave-one-out cross-validation (LOO-CV).

LOO log predictive probability

Predict y_i from the model trained on $\mathcal{D}_{-i} = \mathcal{D} \setminus \{(\mathbf{x}_i, y_i)\}$:

$$L_{\mathrm{LOO}}(X, \mathbf{y}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \log p(y_i \mid \mathbf{x}_i, \mathcal{D}_{-i}, \boldsymbol{\theta}).$$

Remark

Naively, LOO-CV requires n separate model fits ($\mathcal{O}(n^4)$).

Efficient LOO-CV for GP Regression

Key result

For GP regression, each leave-one-out predictive distribution is Gaussian with:

$$\mu_i = y_i - \frac{[K_y^{-1}\mathbf{y}]_i}{[K_y^{-1}]_{ii}}, \qquad \sigma_i^2 = \frac{1}{[K_y^{-1}]_{ii}},$$

using the **same** matrix K_y^{-1} for all i.

Computational saving

- Compute K_y^{-1} **once** via Cholesky: $\mathcal{O}(n^3)$.
- Extract all μ_i and σ_i^2: $\mathcal{O}(n^2)$ extra.
- Total cost: $\mathcal{O}(n^3)$ — same as a single GP fit.

LOO-CV Shortcut — Log Pseudo-likelihood

LOO log pseudo-likelihood

$$L_{\mathrm{LOO}} = -\frac{1}{2}\sum_{i=1}^{n}\left[\log\sigma_i^2 + \frac{(y_i-\mu_i)^2}{\sigma_i^2} + \log 2\pi\right].$$

Remarks on Cross-Validation

- Cross-validation can be used with any loss function.
- The squared error loss is by far the most common for regression.
- The negative log probability loss (log pseudo-likelihood) is more informative about uncertainty calibration and is also available for GPs.

Pros of LOO-CV

- Works with any model.
- Efficient for GPs ($\mathcal{O}(n^3)$ shortcut).
- Direct measure of predictive accuracy.

Pros of Marginal Likelihood

- Closed form for GP regression.
- Automatic Occam's razor.
- Enables Bayesian model comparison.

Overview

1. Model Selection Problems

2. Bayesian Model Selection

3. Cross-Validation for Model Selection

4. Automatic Relevance Determination

5. Model Training for GP Regression/Classification
   5.1 GP Regression Training
   5.2 GP Classification Training

From Isotropic to ARD Kernels

Isotropic SE Kernel (single length-scale ℓ)

$$k_{\text{SE}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right)$$

All input dimensions are treated equally — one parameter controls "smoothness" in every direction.

ARD SE Kernel (one length-scale ℓ_d per dimension)

$$k_{\text{ARD}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{1}{2}\sum_{d=1}^{D} \frac{(x_d - x_d')^2}{\ell_d^2}\right)$$

Each ℓ_d independently controls how fast the function varies along dimension d.

Hyperparameters: $\boldsymbol{\theta} = \{\sigma_f^2, \ell_1, \ldots, \ell_D, \sigma_n^2\}$ — dimensionality increases with D.

ARD: Interpretation of Length-Scales

What does ℓ_d tell us?

- **Small** ℓ_d: the function varies *rapidly* along dimension $d \rightarrow$ dimension d is relevant.
- **Large** ℓ_d: the function barely changes along dimension $d \rightarrow$ dimension d is irrelevant.

Automatic Relevance Determination (ARD)

After optimising $\boldsymbol{\theta}$, inspect $\hat{\ell}_d$:

$$\hat{\ell}_d \to \infty \quad \Longrightarrow \quad \text{dimension } d \text{ can be } \textbf{discarded}.$$

ARD provides a principled, probabilistic approach to **feature selection** without extra model comparisons.

ARD: How Do We Learn the ℓ_d?

Optimisation objective

Treat the ARD length-scales as ordinary hyper-parameters and optimise

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \log p(\mathbf{y} \mid X, \boldsymbol{\theta}), \qquad \boldsymbol{\theta} = \{\sigma_f^2, \ell_1, \ldots, \ell_D, \sigma_n^2\}.$$

So each ℓ_d is learned by maximising the marginal likelihood on the training data.

Gradient for each dimension

For the ARD kernel,

$$\frac{\partial k_{\text{ARD}}}{\partial \ell_d}(\mathbf{x}, \mathbf{x}') = k_{\text{ARD}}(\mathbf{x}, \mathbf{x}') \, \frac{(x_d - x'_d)^2}{\ell_d^3}.$$

Insert this into the general GP gradient formula:

$$\frac{\partial \mathcal{L}}{\partial \ell_d} = \frac{1}{2} \operatorname{tr}\left((\boldsymbol{\alpha}\boldsymbol{\alpha}^\top - K_y^{-1}) \frac{\partial K_y}{\partial \ell_d} \right), \qquad \boldsymbol{\alpha} = K_y^{-1}\mathbf{y}.$$

ARD: Gradient and Optimisation

Practical implication

- **Gradient pushes irrelevant** $\ell_d \to \infty$: the marginal likelihood improves when an irrelevant dimension is "switched off".
- Same $O(n^3)$ cost per gradient step regardless of D — ARD scales to high-dimensional problems.
- Works equally for GP regression and classification (Laplace/EP) by substituting the appropriate approximate log marginal likelihood.

Overview

1. Model Selection Problems

2. Bayesian Model Selection

3. Cross-Validation for Model Selection

4. Automatic Relevance Determination

5. Model Training for GP Regression/Classification

5.1 GP Regression Training

5.2 GP Classification Training

Why GP Regression is Special

The Bayesian challenge

For most models, computing $p(\mathbf{y}|X,\boldsymbol{\theta}) = \int p(\mathbf{y}|\mathbf{f})p(\mathbf{f}|X,\boldsymbol{\theta})d\mathbf{f}$ is intractable.

GP regression advantage

Gaussian process regression with Gaussian noise is a rare exception:

- The marginal likelihood is analytically tractable (closed-form!);
- No need for approximate inference or sampling;
- Models remain highly flexible (nonparametric).

Key benefit

Maximizing the log marginal likelihood optimizes hyperparameters *and* implements automatic Occam's razor, all in $\mathcal{O}(n^3)$ time.

Log Marginal Likelihood for GP Regression

The formula

The log marginal likelihood (also called the log evidence) has a closed form:

$$\mathcal{L}(\boldsymbol{\theta}) \triangleq \log p(\mathbf{y}|X, \boldsymbol{\theta}) = -\tfrac{1}{2}\mathbf{y}^T K_y^{-1}\mathbf{y} - \tfrac{1}{2}\log|K_y| - \tfrac{n}{2}\log 2\pi \tag{4}$$

where $K_y = K_f + \sigma_n^2 I$.

Interpretation: Three components

1. Data fit term ($-\frac{1}{2}\mathbf{y}^T K_y^{-1}\mathbf{y}$): How well does the model explain the observations? *Larger is better.*
2. Complexity penalty ($-\frac{1}{2}\log|K_y|$): How complex/flexible is the model? Automatic Occam's razor. *Penalizes overfitting.*
3. Normalization constant ($-\frac{n}{2}\log 2\pi$): Independent of $\boldsymbol{\theta}$; can be ignored during optimization.

Visualizing the Trade-off

Setup: Data from a GP with SE kernel, true parameters $(\ell, \sigma_f, \sigma_n) = (1, 1, 0.1)$.

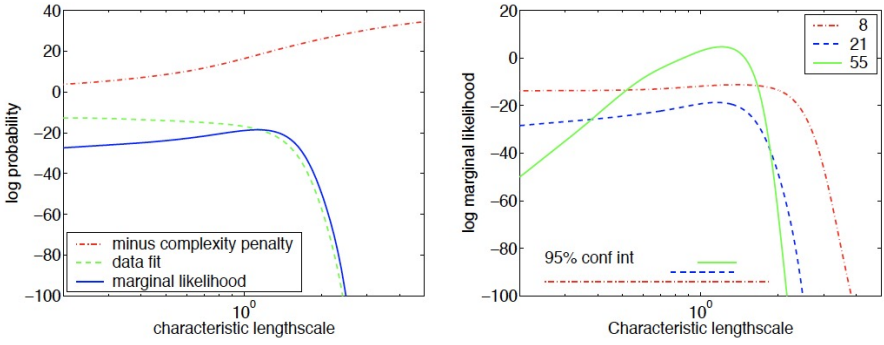


Figure: As ℓ increases, data fit $\downarrow$ (model too smooth), complexity penalty $\uparrow$ (model simpler). Marginal likelihood peaks near the true value $\ell \approx 1$. More data $\Rightarrow$ sharper peak (more confident about optimal ℓ)

Optimizing Hyperparameters via Gradients

Goal: Find $\hat{\theta} = \arg\max_{\theta} \mathcal{L}(\theta)$ using gradient-based optimization.

Key gradient formula

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \frac{1}{2}\mathrm{tr}\left(\left(\boldsymbol{\alpha}\boldsymbol{\alpha}^T - K_y^{-1}\right)\frac{\partial K_y}{\partial \theta_j}\right) \tag{5}$$

where $\boldsymbol{\alpha} = K_y^{-1}\mathbf{y}$.

Computational cost

- **One-time:** Cholesky decomposition of K_y takes $\mathcal{O}(n^3)$
- **Per hyperparameter:** Computing each $\frac{\partial \mathcal{L}}{\partial \theta_j}$ takes $\mathcal{O}(n^2)$
- **Practical:** Use L-BFGS or conjugate gradients; multiple random restarts

Gradient Derivation (Optional Details)

Goal: Derive $\frac{\partial \mathcal{L}}{\partial \theta_j} = \frac{1}{2}\mathrm{tr}\left((\boldsymbol{\alpha}\boldsymbol{\alpha}^\top - K_y^{-1})\frac{\partial K_y}{\partial \theta_j}\right)$.

Step 1 — Data fit term:

$$\frac{\partial}{\partial \theta_j}\left(-\frac{1}{2}\mathbf{y}^T K_y^{-1}\mathbf{y}\right) = \frac{1}{2}\mathbf{y}^T K_y^{-1}\frac{\partial K_y}{\partial \theta_j}K_y^{-1}\mathbf{y} = \frac{1}{2}\mathrm{tr}\left(\boldsymbol{\alpha}\boldsymbol{\alpha}^\top \frac{\partial K_y}{\partial \theta_j}\right)$$

where $\boldsymbol{\alpha} = K_y^{-1}\mathbf{y}$ and we used the trace trick: $\mathbf{a}^T B\mathbf{a} = \mathrm{tr}(\mathbf{a}\mathbf{a}^T B)$.

Step 2 — Complexity penalty:

$$\frac{\partial}{\partial \theta_j}\left(-\frac{1}{2}\log|K_y|\right) = -\frac{1}{2}\mathrm{tr}\left(K_y^{-1}\frac{\partial K_y}{\partial \theta_j}\right)$$

using the matrix identity $\frac{\partial \log|A|}{\partial \theta} = \mathrm{tr}(A^{-1}\frac{\partial A}{\partial \theta})$.

Step 3 — Combine: Sum the two terms and factor out the trace. □

Computing Kernel Derivatives: SE Example

SE kernel: $k(\mathbf{x}_p, \mathbf{x}_q; \ell, \sigma_f^2) = \sigma_f^2 \exp\left(-\frac{r_{pq}^2}{2\ell^2}\right)$ where $r_{pq} = \|\mathbf{x}_p - \mathbf{x}_q\|$.

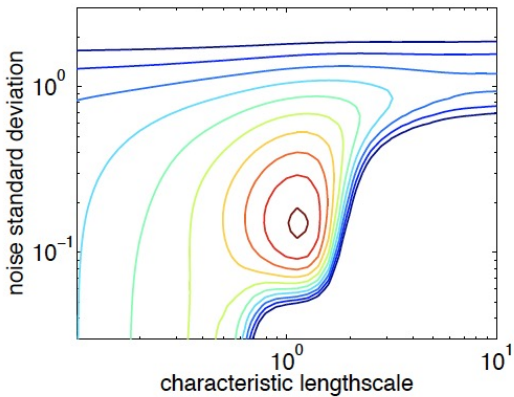
Partial derivatives needed for gradient computation

$$\frac{\partial k}{\partial \sigma_f^2} = \frac{k}{\sigma_f^2} \quad \text{(signal variance)}, \quad \frac{\partial k}{\partial \ell} = k \cdot \frac{r_{pq}^2}{\ell^3} \quad \text{(length-scale)}, \quad \frac{\partial K_y}{\partial \sigma_n^2} = I \quad \text{(noise variance)}.$$

Implementation tip

- Compute these element-wise from the stored K_f matrix
- No need to re-evaluate $k(\cdot, \cdot)$ at each gradient step
- For ARD kernels, compute $\frac{\partial k}{\partial \ell_d}$ for each dimension d

Optimization Landscape



2D optimization over (ℓ, σ_n):

- global optimum near the true
- Smooth, unimodal landscape

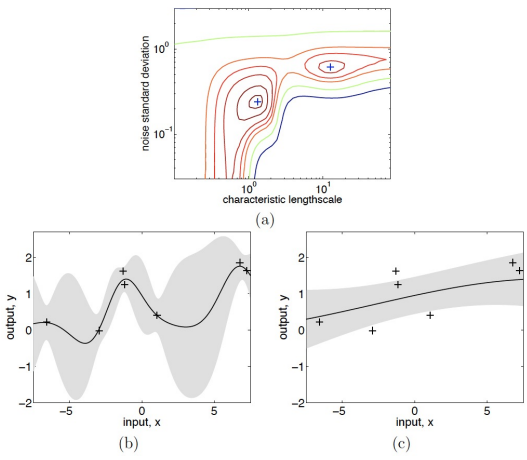
Warning

The marginal likelihood **can** have multiple local optima, especially with:

- Small datasets
- Complex kernels
- Many hyperparameters

$\Rightarrow$ Use multiple random restarts!

Example: Multiple Local Optima



Setup: Only $n = 7$ observations; optimize (ℓ, σ_n) with fixed $\sigma_f = 1$.

Two local optima:

- **(b) Low noise, complex:** Small σ_n, small $\ell \rightarrow$ wiggly function, fits data tightly
- **(c) High noise, simple:** Large σ_n, large $\ell \rightarrow$ smooth function, explains variation as noise

Lesson: With limited data, multiple explanations can be plausible. Try multiple initializations and compare final likelihoods.

Advanced Example: Composite Kernels (Optional)

For complex real-world data (e.g., CO_2 concentration), combine multiple kernels:

Composite kernel structure

$$k(x, x') = k_1(x, x') + k_2(x, x') + k_3(x, x') + k_4(x, x')$$

Component interpretation:

1. k_1: SE kernel $\rightarrow$ long-term smooth trend
2. k_2: Local periodic kernel $\rightarrow$ seasonal variation (period $\approx$ 1 year)
3. k_3: Rational quadratic (RQ) kernel $\rightarrow$ medium-term irregularities
4. k_4: SE + noise $\rightarrow$ short-term fluctuations and measurement noise

Key insight: Marginal likelihood optimization automatically balances complexity vs. fit across all components.

Extending to GP Classification

The challenge

For classification with non-Gaussian likelihoods (e.g., logistic, probit), the marginal likelihood $p(\mathbf{y}|X, \boldsymbol{\theta})$ is **no longer analytically tractable**.

Solution: Use approximations

We approximate $p(\mathbf{y}|X, \boldsymbol{\theta})$ and optimize the approximate marginal likelihood:

- **Laplace approximation:** Gaussian approximation at the posterior mode
- **EP (Expectation Propagation):** More accurate moment matching (optional)

Training procedure

1. Compute approximate log marginal likelihood $\log q(\mathbf{y}|X, \boldsymbol{\theta})$
2. Compute gradients w.r.t. hyperparameters $\boldsymbol{\theta}$
3. Optimize using gradient ascent / L-BFGS

Laplace Approximation: Gradient Derivation

Recall: Laplace gives the approximate log marginal likelihood

$$\log q(\mathbf{y}|X,\boldsymbol{\theta}) = -\frac{1}{2}\hat{\mathbf{f}}^T K^{-1}\hat{\mathbf{f}} + \log p(\mathbf{y}|\hat{\mathbf{f}}) - \frac{1}{2}\log|B|, \tag{6}$$

where $B = I + WK$, $W = -\nabla\nabla \log p(\mathbf{y}|\hat{\mathbf{f}})$, and $\hat{\mathbf{f}}$ is the posterior mode.

Key complication

Both K and $\hat{\mathbf{f}}$ depend on $\boldsymbol{\theta}$! We need to account for:

1. Explicit dependence: $K(\boldsymbol{\theta})$ appears directly in the formula
2. Implicit dependence: $\hat{\mathbf{f}}(\boldsymbol{\theta})$ changes when $\boldsymbol{\theta}$ changes

Total derivative:

$$\frac{\partial \log q}{\partial \theta_j} = \underbrace{\left.\frac{\partial \log q}{\partial \theta_j}\right|_{\text{explicit}}}_{\text{direct dependence}} + \underbrace{\sum_{i=1}^{n} \frac{\partial \log q}{\partial \hat{f}_i}\frac{\partial \hat{f}_i}{\partial \theta_j}}_{\text{mode shift}} \tag{7}$$

Laplace Gradient: Complete Formula

1. Explicit term (direct K dependence)

$$\left.\frac{\partial \log q}{\partial \theta_j}\right|_{\text{explicit}} = \frac{1}{2}\hat{\mathbf{f}}^T K^{-1}\frac{\partial K}{\partial \theta_j}K^{-1}\hat{\mathbf{f}} - \frac{1}{2}\mathrm{tr}\left((W^{-1}+K)^{-1}\frac{\partial K}{\partial \theta_j}\right) \tag{8}$$

2. Implicit term (mode shift correction)

Key simplification: Since $\hat{\mathbf{f}}$ is at the posterior mode, the first two terms of eq. (6) have zero gradient w.r.t. $\hat{\mathbf{f}}$. Only the log-determinant contributes:

$$\frac{\partial \log q}{\partial \hat{f}_i} = -\frac{1}{2}\mathrm{tr}\left(B^{-1}K\frac{\partial W}{\partial \hat{f}_i}\right) \tag{9}$$

Then use the implicit function theorem to find $\frac{\partial \hat{\mathbf{f}}}{\partial \theta_j}$ from the optimality condition.

Summary

Key Takeaways from This Lecture:

1. Model Selection Framework

- Balance model fit vs. complexity to avoid underfitting/overfitting
- Bayesian hierarchical approach: inference over $\mathbf{f}$, $\boldsymbol{\theta}$, and $\mathcal{H}$
- Occam's Razor: marginal likelihood $p(\mathbf{y}|X, \boldsymbol{\theta})$ automatically penalizes complexity

2. GP Regression Training — Closed-form marginal likelihood:

$$\mathcal{L}(\boldsymbol{\theta}) = \underbrace{-\frac{1}{2}\mathbf{y}^T K_y^{-1}\mathbf{y}}_{\text{data fit}} - \underbrace{\frac{1}{2}\log|K_y|}_{\text{complexity}} - \frac{n}{2}\log 2\pi$$

Gradient-based optimization: $O(n^3)$ Cholesky + $O(n^2)$ per hyperparameter.

Summary

Key Takeaways from This Lecture:

3. Model Selection Techniques

- **LOO-CV:** $O(n^3)$ shortcut for GPs (same cost as single fit); robust to misspecification
- **ARD:** One ℓ_d per dimension; large $\hat{\ell}_d \to \infty$ indicates irrelevance (automatic feature selection)

4. GP Classification — Non-Gaussian likelihood $\Rightarrow$ use Laplace (Gaussian at mode) or EP (more accurate) approximations;

$$\frac{\partial \log q}{\partial \theta_j} = \underbrace{\left.\frac{\partial \log q}{\partial \theta_j}\right|_{\text{explicit}}}_{\text{direct dependence}} + \underbrace{\sum_{i=1}^{n} \frac{\partial \log q}{\partial \hat{f}_i} \frac{\partial \hat{f}_i}{\partial \theta_j}}_{\text{mode shift}}$$

The End